TOTALRECALLS POWER USER GUIDE

Building a Local Vector Search & SQL Container

Step-by-Step Blueprint for Indexing Your Top 8 AI Providers into a Private Local RAG Engine

1. Executive Overview: The Top 8 Provider Advantage

Standard AI interfaces isolate your work in proprietary web sidebars. While desktop competitors charge fees and limit coverage to only 4 providers, **TotalRecalls exports conversations from the Top 8 AI providers in the world**: **ChatGPT, Claude, Perplexity, Gemini, Grok, DeepSeek, Mistral, and Qwen**. By outputting every conversation as paired **.md** (Markdown) and **.json** (Structured JSON) files into C:\TotalRecalls\Library\, TotalRecalls enables you to transform your local hard drive into a private, offline **Vector Database and SQL Query Container**.

Feature / Spec	TotalRecalls Desktop	Closest Competitor (AI Chat Importer)	Browser Extensions
Supported Providers	TOP 8 (ChatGPT, Claude, Perplexity, Gemini, Grok, DeepSeek, Mistral, Qwen)	4 Providers Only	1 or 2 Providers
Local Export Formats	Paired .md (Markdown) + .json (Structured Data)	Proprietary / HTML Archive	Raw HTML or TXT
Vector & SQL Indexing	100% Native Compatible (Standard Schema)	Requires Export Conversion	Not Supported
Data Privacy	100% Local PC (No Cloud, Session Tokens Local)	Varies	Third-Party Cloud Processing

Table 1: *Strategic Differentiator Matrix — 8-Provider Multi-Model Infrastructure vs Competitors.*

2. Step 1: Exporting Your 8 AI Models via TotalRecalls

Open **TotalRecalls** on your Windows PC and execute an export across your active AI providers. TotalRecalls creates organized subfolders inside your main library root path:

■■ DESKTOP SNAPSHOT: FILE EXPLORER VIEW (C:\TOTALRECALLS\LIBRARY\)

```
■ C:\TotalRecalls\Library\
   ■■■ ■ chatgpt\      ■■■ 2026-10-15-async-python-architecture.md  [+.json]
   ■■■ ■ claude\       ■■■ 2026-10-18-database-refactoring-plan.md  [+.json]
   ■■■ ■ perplexity\   ■■■ 2026-10-20-market-research-citations.md  [+.json]
   ■■■ ■ gemini\       ■■■ 2026-10-21-google-takeout-import.md      [+.json]
   ■■■ ■ grok\         ■■■ 2026-10-22-xai-analysis-script.md         [+.json]
   ■■■ ■ deepseek\     ■■■ 2026-10-23-cpp-memory-optimization.md    [+.json]
   ■■■ ■ mistral\      ■■■ 2026-10-24-le-chat-prompts-archive.md    [+.json]
   ■■■ ■ qwen\         ■■■ 2026-10-25-qwen-local-model-bench.md     [+.json]
```

3. Step 2: Building the Local SQL Container (SQLite Ingestion)

To enable relational filtering, date ranges, and SQL metadata queries, run this Python script to parse all `.json` files across the 8 provider subfolders into a single local SQLite database file (totalrecalls_chats.db):

```
import os, json, sqlite3
# Initialize SQLite Container
conn = sqlite3.connect('C:\TotalRecalls\totalrecalls_chats.db')
cursor = conn.cursor()
cursor.execute('''CREATE TABLE IF NOT EXISTS chats (
    id TEXT PRIMARY KEY, provider TEXT, title TEXT,
    created_at TEXT, message_count INTEGER, file_path TEXT
)''')
library_path = 'C:\TotalRecalls\Library'
for root, dirs, files in os.walk(library_path):
    for f in files:
        if f.endswith('.json'):
            full_path = os.path.join(root, f)
            with open(full_path, 'r', encoding='utf-8') as jf:
                data = json.load(jf)
                cursor.execute('''INSERT OR REPLACE INTO chats VALUES (?,?,?,?,?,?)''', (
                    data.get('id'), data.get('provider'), data.get('title'),
                    data.get('created_at'), len(data.get('messages', [])), full_path
                ))
conn.commit()
print("SQL Container Built Successfully!")
```

■■ DESKTOP SNAPSHOT: TERMINAL OUTPUT - LOCAL SQL DATABASE BUILT

```
COMMAND PROMPT / POWERSHELL > python build_sql_container.py
[SUCCESS] Scanning 8 Provider Subfolders in C:\TotalRecalls\Library\...
[PARSED] 142 ChatGPT chats | 89 Claude chats | 64 Perplexity threads
[PARSED] 38 Gemini Takeout files | 45 Grok chats | 52 DeepSeek logs
[PARSED] 21 Mistral chats | 30 Qwen sessions
[DATABASE] Total 481 conversations written to totalrecalls_chats.db (1.8 MB)
```

4. Step 3: Building the Offline Vector Search Engine

Semantic search requires converting your Markdown text into numerical vector embeddings. By pairing **LanceDB** or **ChromaDB** with a lightweight local model (like sentence-transformers/all-MiniLM-L6-v2), your vector engine runs 100% offline on your PC's CPU or GPU:

```
import lancedb
from sentence_transformers import SentenceTransformer
import glob
# Load local embedding model (100% offline)
embedder = SentenceTransformer('all-MiniLM-L6-v2')
db = lancedb.connect('C:\TotalRecalls\vector_store')
documents = []
for filepath in glob.glob('C:\TotalRecalls\Library\**\*.md', recursive=True):
    with open(filepath, 'r', encoding='utf-8') as f:
        content = f.read()
        provider = filepath.split('\')[-2]
        # Chunk text into 500-char blocks
        for idx, chunk in enumerate([content[i:i+500] for i in range(0, len(content), 500)]):
            vector = embedder.encode(chunk).tolist()
            documents.append({'id': f"{filepath}_{idx}", 'vector': vector, 'text': chunk, 'provider': provide
r, 'path': filepath})
table = db.create_table("ai_knowledge_base", data=documents, mode="overwrite")
print(f"Indexed {len(documents)} chunks across Top 8 Providers into LanceDB Vector Store!")
```

■■ DESKTOP SNAPSHOT: VS CODE TERMINAL - VECTOR INDEXING COMPLETED

```
VS CODE TERMINAL > python build_vector_index.py
[MODEL] Loaded sentence-transformers/all-MiniLM-L6-v2 on Local CPU/GPU
[INDEXING] Chunking .md files from ChatGPT, Claude, Perplexity, Gemini, Grok, DeepSeek, Mistral, Qwen...
[VECTOR DB] Generated 3,412 vector embeddings (384 dimensions)
[SAVED] Vector Index written to C:\TotalRecalls\vector_store\ai_knowledge_base.lance
```

5. Step 4: Querying Across All 8 Providers Simultaneously

Now you can perform natural language semantic queries across every AI conversation you have ever had, regardless of which model or sidebar originated it:

```
# Run natural language query across all 8 provider histories
query = "asynchronous queue worker error handling retry pattern"
query_vector = embedder.encode(query).tolist()
results = table.search(query_vector).limit(3).to_pandas()
for idx, row in results.iterrows():
    print(f"[{row['provider'].upper()}] Match (Score: {row['_distance']:.4f}):")
    print(f" File: {row['path']}")
    print(f" Text: {row['text'][:150]}...\n")
```

■■ DESKTOP SNAPSHOT: COMMAND PROMPT - CROSS-PROVIDER SEMANTIC SEARCH RESULTS

```
COMMAND PROMPT > python search_my_ai.py "asynchronous queue error handling"
--------------------------------------------------------------------------------
[MATCH 1 - DEEPSEEK] (Score: 0.142)
 File: C:\TotalRecalls\Library\deepseek\2026-09-12-python-asyncio-tasks.md
 Snippet: "...when task worker raises Exception, use asyncio.sleep(retry) with exponential backoff..."
--------------------------------------------------------------------------------
[MATCH 2 - CLAUDE] (Score: 0.188)
 File: C:\TotalRecalls\Library\claude\2026-08-04-backend-queue-design.md
 Snippet: "...robust retry logic requires dead-letter queuing for persistent failures..."
--------------------------------------------------------------------------------
[MATCH 3 - CHATGPT] (Score: 0.210)
 File: C:\TotalRecalls\Library\chatgpt\2026-05-19-redis-celery-setup.md
```

6. Step 5: Connecting Your Container to Obsidian & Local AI Agents

Because TotalRecalls outputs plain files and standard SQL/Vector containers, you can directly integrate your entire multi-provider archive with your favorite local desktop tools:

• **Obsidian Vault Integration**: Simply set your Obsidian Vault path to C:\TotalRecalls\Library\. All 8 provider subfolders immediately appear as native markdown notes with full graph connectivity.
• **Local LLM RAG (Ollama / AnythingLLM)**: Point AnythingLLM or Ollama RAG extension at C:\TotalRecalls\vector_store\ to chat with your entire past AI knowledge base offline.

> **SUMMARY VALUE PROPOSITION:**
> TotalRecalls doesn't just download files — it unlocks complete data sovereignty. By capturing the **Top 8 AI providers** in open Markdown and JSON formats, TotalRecalls turns your scattered chat history into a permanent, searchable, private local database.